Experiment 4 : Analyze ***Superstore_USA.csv*** dataset and answer given questions.

> > ## load dataset

In [2]:
```
import pandas as pd
import numpy as np

# read file
df = pd.read_csv('Superstore_USA.csv',encoding='cp1252')
df.head()
```

Out[2]:

	Row ID	Order Priority	Discount	Unit Price	Shipping Cost	Customer ID	Customer Name	Ship Mode	Customer Segment
0	18606	Not Specified	0.01	2.88	0.50	2	Janice Fletcher	Regular Air	Corporate
1	20847	High	0.01	2.84	0.93	3	Bonnie Potter	Express Air	Corporate
2	23086	Not Specified	0.03	6.68	6.15	3	Bonnie Potter	Express Air	Corporate
3	23087	Not Specified	0.01	5.68	3.60	3	Bonnie Potter	Regular Air	Corporate
4	23088	Not Specified	0.00	205.99	2.50	3	Bonnie Potter	Express Air	Corporate

5 rows × 24 columns

In [3]:
```
# check Structure
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9426 entries, 0 to 9425
Data columns (total 24 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Row ID                9426 non-null   int64
 1   Order Priority        9426 non-null   object
 2   Discount              9426 non-null   float64
 3   Unit Price            9426 non-null   float64
 4   Shipping Cost         9426 non-null   float64
 5   Customer ID           9426 non-null   int64
 6   Customer Name         9426 non-null   object
 7   Ship Mode             9426 non-null   object
 8   Customer Segment      9426 non-null   object
 9   Product Category      9426 non-null   object
 10  Product Sub-Category  9426 non-null   object
 11  Product Container     9426 non-null   object
 12  Product Name          9426 non-null   object
 13  Product Base Margin   9354 non-null   float64
 14  Region                9426 non-null   object
 15  State or Province     9426 non-null   object
 16  City                  9426 non-null   object
 17  Postal Code           9426 non-null   int64
 18  Order Date            9426 non-null   object
 19  Ship Date             9426 non-null   object
 20  Profit                9426 non-null   float64
 21  Quantity ordered new  9426 non-null   int64
 22  Sales                 9426 non-null   float64
 23  Order ID              9426 non-null   int64
dtypes: float64(6), int64(5), object(13)
memory usage: 1.7+ MB
```

In [4]:
```
df.columns
```

Out[4]:
```
Index(['Row ID', 'Order Priority', 'Discount', 'Unit Price', 'Shipping Cost',
       'Customer ID', 'Customer Name', 'Ship Mode', 'Customer Segment',
       'Product Category', 'Product Sub-Category', 'Product Container',
       'Product Name', 'Product Base Margin', 'Region', 'State or Province',
       'City', 'Postal Code', 'Order Date', 'Ship Date', 'Profit',
       'Quantity ordered new', 'Sales', 'Order ID'],
      dtype='object')
```

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
Row ID                   0
Order Priority           0
Discount                 0
Unit Price               0
Shipping Cost            0
Customer ID              0
Customer Name            0
Ship Mode                0
Customer Segment         0
Product Category         0
Product Sub-Category     0
Product Container        0
Product Name             0
Product Base Margin     72
Region                   0
State or Province        0
City                     0
Postal Code              0
Order Date               0
Ship Date                0
Profit                   0
Quantity ordered new     0
Sales                    0
Order ID                 0
dtype: int64
```

In [6]:
```
# convert date columns to datetime
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Ship Date'] = pd.to_datetime(df['Ship Date'])

#remove duplicate rows
df = df.drop_duplicates()      #our data does't have missing values

# drop missing values
df = df.dropna()
```

In [7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 9354 entries, 0 to 9425
Data columns (total 24 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Row ID                9354 non-null   int64
 1   Order Priority        9354 non-null   object
 2   Discount              9354 non-null   float64
 3   Unit Price            9354 non-null   float64
 4   Shipping Cost         9354 non-null   float64
 5   Customer ID           9354 non-null   int64
 6   Customer Name         9354 non-null   object
 7   Ship Mode             9354 non-null   object
 8   Customer Segment      9354 non-null   object
 9   Product Category      9354 non-null   object
 10  Product Sub-Category  9354 non-null   object
 11  Product Container     9354 non-null   object
 12  Product Name          9354 non-null   object
 13  Product Base Margin   9354 non-null   float64
 14  Region                9354 non-null   object
 15  State or Province     9354 non-null   object
 16  City                  9354 non-null   object
 17  Postal Code           9354 non-null   int64
 18  Order Date            9354 non-null   datetime64[ns]
 19  Ship Date             9354 non-null   datetime64[ns]
 20  Profit                9354 non-null   float64
 21  Quantity ordered new  9354 non-null   int64
 22  Sales                 9354 non-null   float64
 23  Order ID              9354 non-null   int64
dtypes: datetime64[ns](2), float64(6), int64(5), object(11)
memory usage: 1.8+ MB
```

1. Try to find out how many unique customers that we have

In [8]:
```
print("Total unique customers: ",df['Customer ID'].nunique())
```

```
Total unique customers:  2694
```

2. Try to find out in how many regions we are selling a product and who is a manager for a respective region

In [9]:
```
df['Region'].unique()
```

Out[9]:
```
array(['Central', 'West', 'East', 'South'], dtype=object)
```

In [10]:
```
region = df['Region'].drop_duplicates()
print("regions: ",region)
# as there is no manager field, managers of region can't be determined
```

```
regions:  0      Central
1         West
13        East
113      South
Name: Region, dtype: object
```

3. Find out how many different shipment modes that we have and what is a percentage usability of all the shipment

mode with respect to dataset

In [11]:
```
shipment_counts = df['Ship Mode'].value_counts()
print(shipment_counts)
```

```
Ship Mode
Regular Air       6992
Delivery Truck    1266
Express Air       1096
Name: count, dtype: int64
```

In [12]:
```
# calculate percentage usage
shipment_percentage = (shipment_counts / len(df)) * 100

result = pd.DataFrame({
    'Count': shipment_counts,
    'Percentage (%)': shipment_percentage
})

print(result)
```

```
                Count  Percentage (%)
Ship Mode
Regular Air      6992       74.748771
Delivery Truck   1266       13.534317
Express Air      1096       11.716913
```

4. Create a new column and try to find out a difference between order date and shipment date

In [13]:
```
df['Shipment Duration'] = (df['Ship Date'] - df['Order Date']).dt.days

# Preview
df[['Order ID', 'Order Date', 'Ship Date', 'Shipment Duration']].head()
```

Out[13]:

	Order ID	Order Date	Ship Date	Shipment Duration
0	88525	2012-05-28	2012-05-30	2
1	88522	2010-07-07	2010-07-08	1
2	88523	2011-07-27	2011-07-28	1
3	88523	2011-07-27	2011-07-28	1
4	88523	2011-07-27	2011-07-27	0

5. Base on question number 8 find out for which order id we have shipment duration more than 10 days

In [14]:
```
orders_gt_10 = df[df['Shipment Duration'] > 10]['Order ID'].unique()

print("Orders with shipment duration > 10 days:")
print(orders_gt_10)
```

```
Orders with shipment duration > 10 days:
[87215 86318 87957 19556 86177 88223 88352 87572 91294 86434 86436 87300
 19841]
```

6. Try to find out a list of a returned order which shipment duration was more than 15 days and find out that region manager as well

In [15]:
```
# returned data is not available
```

7. Gorup by region and find out which region is more profitable

In [16]:
```
region_profit = df.groupby('Region')['Profit'].sum().sort_values(ascending=False)

print(region_profit)
```

```
Region
Central    511534.461466
East       369034.948607
West       299599.064651
South      100322.369321
Name: Profit, dtype: float64
```

8. Try to find out overall in which country we are giving more discount

In [27]:
```
country_discount = df.groupby('State or Province')['Discount'].sum().sort_values(
```

In [18]:
```
top_state = country_discount.idxmax()
print("State with highest discount:", top_state)
```

```
State with highest discount: California
```

9. Give me a list of unique postal code

In [19]:
```
postal_codes = df['Postal Code'].unique()

print(postal_codes)
```

```
[60101 98221 91776 ... 61832 62521 26554]
```

In [20]:
```
print("Total Unique Postal Codes:", df['Postal Code'].nunique())
```

```
Total Unique Postal Codes: 1696
```

10. Which customer segment is more profitable find it out.

In [21]:
```
segment_profit = df.groupby('Customer Segment')['Profit'].sum().sort_values(ascen

print(segment_profit)
```

```
Customer Segment
Corporate         503420.896285
Small Business    300128.147362
Home Office       274723.080009
Consumer          202218.720389
Name: Profit, dtype: float64
```

In [22]:
```
top_segment = segment_profit.idxmax()
print("Most Profitable Segment:", top_segment)
```

```
Most Profitable Segment: Corporate
```

11. Try to find out the 10th most loss-making product category.

In [23]:
```
category_profit = df.groupby('Product Category')['Profit'].sum().sort_values()
print(category_profit)
```

```
Product Category
Furniture          140343.135390
Office Supplies    457049.823913
Technology         683097.884742
Name: Profit, dtype: float64
```

In [24]:
```
if len(category_profit) >= 10:
    tenth_loss = category_profit.iloc[9]
    print("10th most loss-making category:", tenth_loss)
else:
    print("Not enough categories to find 10th loss-making")
```

```
Not enough categories to find 10th loss-making
```

12. Try to find out 10 top products with highest margins.

In [25]:
```
df['Margin'] = df['Profit'] / df['Sales']
```

In [26]:
```
top_products = df.groupby('Product Name')['Margin'].mean().sort_values(ascending=

print(top_products)
```

```
Product Name
Avery White Multi-Purpose Labels                                    83.290983
Rediform S.O.S. Phone Message Books                                 63.078257
Telephone Message Books with Fax/Mobile Section, 4 1/4" x 6"        35.920390
Xerox 1981                                                          16.523400
Advantus Panel Wall Acrylic Frame                                   14.271535
Rogers Handheld Barrel Pencil Sharpener                             13.894232
Plymouth Boxed Rubber Bands by Plymouth                             13.106785
Avery Hi-Liter Comfort Grip Fluorescent Highlighter, Yellow Ink     12.914191
Avery 506                                                           12.315710
GBC Plastic Binding Combs                                           11.369849
Name: Margin, dtype: float64
```